

**Московский государственный технический
университет им. Н.Э. Баумана**

Факультет информатика и системы управления
Кафедра системы обработки информации и управления

Курс «Парадигмы и конструкции языков программирования»
Отчет по лабораторной работы №3-4

Выполнил:
студент группы ИУ5-32Б:
Кунев В.
Подпись и дата:

Проверил:
преподаватель каф. ИУ5
Нардид А.Н.
Подпись и дата:

Москва, 2025 г.

Описание задания

Цель лабораторной работы: изучение возможностей функционального программирования в языке Python.

Задание:

Задание лабораторной работы состоит из решения нескольких задач.

Файлы, содержащие решения отдельных задач, должны располагаться в пакете `lab_python_fr`. Решение каждой задачи должно располагаться в отдельном файле.

При запуске каждого файла выдаются тестовые результаты выполнения соответствующего задания.

Задача 1 (файл `field.py`)

Необходимо реализовать генератор `field`. Генератор `field` последовательно выдает значения ключей словаря. Пример:

```
goods = [
    {'title': 'Ковер', 'price': 2000, 'color': 'green'},
    {'title': 'Диван для отдыха', 'color': 'black'}
]
```

`field(goods, 'title')` должен выдавать 'Ковер', 'Диван для отдыха'

`field(goods, 'title', 'price')` должен выдавать `{'title': 'Ковер', 'price': 2000}`, `{'title': 'Диван для отдыха'}`

- В качестве первого аргумента генератор принимает список словарей, дальше через `*args` генератор принимает неограниченное количество аргументов.
- Если передан один аргумент, генератор последовательно выдает только значения полей, если значение поля равно `None`, то элемент пропускается.
- Если передано несколько аргументов, то последовательно выдаются словари, содержащие данные элементы. Если поле равно `None`, то оно пропускается. Если все поля содержат значения `None`, то пропускается элемент целиком.

Шаблон для реализации генератора:

```
# Пример:
# goods = [
#     {'title': 'Ковер', 'price': 2000, 'color': 'green'},
#     {'title': 'Диван для отдыха', 'price': 5300, 'color': 'black'}
# ]
# field(goods, 'title') должен выдавать 'Ковер', 'Диван для отдыха'
# field(goods, 'title', 'price') должен выдавать {'title': 'Ковер', 'price':
2000}, {'title': 'Диван для отдыха', 'price': 5300}
```

```
def field(items, *args):  
    assert len(args) > 0  
    # Необходимо реализовать генератор
```

Задача 2 (файл gen_random.py)

Необходимо реализовать генератор gen_random(количество, минимум, максимум), который последовательно выдает заданное количество случайных чисел в заданном диапазоне от минимума до максимума, включая границы диапазона. Пример:

gen_random(5, 1, 3) должен выдать 5 случайных чисел в диапазоне от 1 до 3, например 2, 2, 3, 2, 1

Шаблон для реализации генератора:

```
# Пример:  
# gen_random(5, 1, 3) должен выдать 5 случайных чисел  
# в диапазоне от 1 до 3, например 2, 2, 3, 2, 1  
# Hint: типовая реализация занимает 2 строки  
def gen_random(num_count, begin, end):  
    pass  
    # Необходимо реализовать генератор
```

Задача 3 (файл unique.py)

- Необходимо реализовать итератор Unique(данные), который принимает на вход массив или генератор и итерируется по элементам, пропуская дубликаты.
- Конструктор итератора также принимает на вход именованный bool-параметр ignore_case, в зависимости от значения которого будут считаться одинаковыми строки в разном регистре. По умолчанию этот параметр равен False.
- При реализации необходимо использовать конструкцию **kwargs.
- Итератор должен поддерживать работу как со списками, так и с генераторами.
- Итератор не должен модифицировать возвращаемые значения.

Пример:

```
data = [1, 1, 1, 1, 1, 2, 2, 2, 2, 2]
```

Unique(data) будет последовательно возвращать только 1 и 2.

```
data = gen_random(10, 1, 3)
```

Unique(data) будет последовательно возвращать только 1, 2 и 3.

```
data = ['a', 'A', 'b', 'B', 'a', 'A', 'b', 'B']
```

Unique(data) будет последовательно возвращать только a, A, b, B.

Unique(data, ignore_case=True) будет последовательно возвращать только a, b.

Шаблон для реализации класса-итератора:

```
# Итератор для удаления дубликатов
class Unique(object):
    def __init__(self, items, **kwargs):
        # Нужно реализовать конструктор
        # В качестве ключевого аргумента, конструктор должен принимать bool-
        параметр ignore_case,
        # в зависимости от значения которого будут считаться одинаковыми строки в
        разном регистре
        # Например: ignore_case = True, Абв и АБВ - разные строки
        # ignore_case = False, Абв и АБВ - одинаковые строки, одна из
        которых удалится
        # По-умолчанию ignore_case = False
        pass

    def __next__(self):
        # Нужно реализовать __next__
        pass

    def __iter__(self):
        return self
```

Задача 4 (файл sort.py)

Дан массив 1, содержащий положительные и отрицательные числа.

Необходимо **одной строкой кода** вывести на экран массив 2, которые содержит значения массива 1, отсортированные по модулю в порядке убывания. Сортировку необходимо осуществлять с помощью функции sorted. Пример:

```
data = [4, -30, 30, 100, -100, 123, 1, 0, -1, -4]
```

Вывод: [123, 100, -100, -30, 30, 4, -4, 1, -1, 0]

Необходимо решить задачу двумя способами:

1. С использованием lambda-функции.
2. Без использования lambda-функции.

Шаблон реализации:

```
data = [4, -30, 100, -100, 123, 1, 0, -1, -4]
```

```
if __name__ == '__main__':
    result = ...
    print(result)

    result_with_lambda = ...
    print(result_with_lambda)
```

Задача 5 (файл print_result.py)

Необходимо реализовать декоратор `print_result`, который выводит на экран результат выполнения функции.

- Декоратор должен принимать на вход функцию, вызывать её, печатать в консоль имя функции и результат выполнения, после чего возвращать результат выполнения.
- Если функция вернула список (list), то значения элементов списка должны выводиться в столбик.
- Если функция вернула словарь (dict), то ключи и значения должны выводиться в столбик через знак равенства.

Шаблон реализации:

Здесь должна быть реализация декоратора

```
@print_result
def test_1():
    return 1

@print_result
def test_2():
    return 'iu5'

@print_result
def test_3():
    return {'a': 1, 'b': 2}

@print_result
def test_4():
    return [1, 2]

if __name__ == '__main__':
    print('!!!!!!!!')
    test_1()
    test_2()
    test_3()
    test_4()
```

Результат выполнения:

```
test_1
1
test_2
iu5
test_3
a = 1=
```

```
b = 2
test_4
1
2
```

Задача 6 (файл `cm_timer.py`)

Необходимо написать контекстные менеджеры `cm_timer_1` и `cm_timer_2`, которые считают время работы блока кода и выводят его на экран. Пример:

```
with cm_timer_1():
    sleep(5.5)
```

После завершения блока кода в консоль должно вывестись `time: 5.5` (реальное время может несколько отличаться).

`cm_timer_1` и `cm_timer_2` реализуют одинаковую функциональность, но должны быть реализованы двумя различными способами (на основе класса и с использованием библиотеки `contextlib`).

Задача 7 (файл `process_data.py`)

- В предыдущих задачах были написаны все требуемые инструменты для работы с данными. Применим их на реальном примере.
- В файле [data_light.json](#) содержится фрагмент списка вакансий.
- Структура данных представляет собой список словарей с множеством полей: название работы, место, уровень зарплаты и т.д.
- Необходимо реализовать 4 функции - `f1`, `f2`, `f3`, `f4`. Каждая функция вызывается, принимая на вход результат работы предыдущей. За счет декоратора `@print_result` печатается результат, а контекстный менеджер `cm_timer_1` выводит время работы цепочки функций.
- Предполагается, что функции `f1`, `f2`, `f3` будут реализованы в одну строку. В реализации функции `f4` может быть до 3 строк.
- Функция `f1` должна вывести отсортированный список профессий без повторений (строки в разном регистре считать равными). Сортировка должна игнорировать регистр. Используйте наработки из предыдущих задач.
- Функция `f2` должна фильтровать входной массив и возвращать только те элементы, которые начинаются со слова “программист”. Для фильтрации используйте функцию `filter`.
- Функция `f3` должна модифицировать каждый элемент массива, добавив строку “с опытом Python” (все программисты должны быть знакомы с Python). Пример: Программист C# с опытом Python. Для модификации используйте функцию `map`.

- Функция f4 должна сгенерировать для каждой специальности зарплату от 100 000 до 200 000 рублей и присоединить её к названию специальности. Пример: Программист С# с опытом Python, зарплата 137287 руб. Используйте zip для обработки пары специальность — зарплата.

Шаблон реализации:

```
import json
import sys
# Сделаем другие необходимые импорты

path = None

# Необходимо в переменную path сохранить путь к файлу, который был передан при
запуске сценария

with open(path) as f:
    data = json.load(f)

# Далее необходимо реализовать все функции по заданию, заменив `raise
NotImplemented`
# Предполагается, что функции f1, f2, f3 будут реализованы в одну строку
# В реализации функции f4 может быть до 3 строк

@print_result
def f1(arg):
    raise NotImplemented

@print_result
def f2(arg):
    raise NotImplemented

@print_result
def f3(arg):
    raise NotImplemented

@print_result
def f4(arg):
    raise NotImplemented

if __name__ == '__main__':
    with cm_timer_1():
        f4(f3(f2(f1(data))))
```

Текст программы

`main.py`:

```
#!/usr/bin/env python3
"""
Основной файл для тестирования всех модулей
"""

import sys
import os
from time import sleep

from lab_python_fp import field, gen_random, Unique, cm_timer_1, cm_timer_2
from lab_python_fp.sort import data as data_sort
from lab_python_fp.print_result import test_1, test_2, test_3, test_4

# Добавляем путь к пакету
sys.path.insert(0, os.path.join(os.path.dirname(__file__)))

def test_all_modules():
    """Тестирование всех модулей"""
    print("=== Тестирование всех модулей ===\n")

    # Тестирование field
    print("1. Тестирование field:")

    goods = [
        {"title": "Ковер", "price": 2000, "color": "green"},
        {"title": "Диван для отдыха", "color": "black"},
    ]
    print("field(goods, 'title'):", list(field(goods, "title")))
    print("field(goods, 'title', 'price'):", list(field(goods, "title",
"price"))))
    print()

    # Тестирование gen_random
    print("2. Тестирование gen_random:")

    print("gen_random(5, 1, 3):", list(gen_random(5, 1, 3)))
    print()

    # Тестирование unique
    print("3. Тестирование unique:")

    data = [1, 1, 2, 2, 3, 3]
    print(f"Unique({data}):", list(Unique(data)))
    print()

    # Тестирование sort
```



```

print("4. Тестирование sort:")

result = sorted(data_sort, key=abs, reverse=True)
print("Исходные данные:", data_sort)
print("Отсортировано по модулю:", result)
print()

# Тестирование print_result
print("5. Тестирование print_result:")

test_1()
test_2()
test_3()
test_4()
print()

# Тестирование cm_timer
print("6. Тестирование cm_timer:")

print("cm_timer_1 (1 секунда):")
with cm_timer_1():
    sleep(1)

print("cm_timer_2 (0.5 секунды):")
with cm_timer_2():
    sleep(0.5)
print()

if __name__ == "__main__":
    test_all_modules()

```

`lab\_python\_fp/\_\_init\_\_.py`:

```

"""
Пакет lab_python_fp - Функциональные возможности языка Python
Лабораторная работа №3-4
"""

from .field import field
from .gen_random import gen_random
from .unique import Unique
from .print_result import print_result
from .cm_timer import cm_timer_1, cm_timer_2

__all__ = ["field", "gen_random", "Unique", "print_result", "cm_timer_1",
"cm_timer_2"]

__version__ = "1.0.0"

```

```
__author__ = "Valentin Cunev"
```

```
`lab_python_fp/cm_timer.py`:
```

```
"""
Контекстные менеджеры для измерения времени выполнения блока кода
"""
```

```
import time
from contextlib import contextmanager
```

```
# Реализация на основе класса
# Название класса обусловлено условием задачи
class cm_timer_1: # pylint: disable=invalid-name
```

```
    def __enter__(self):
        self.start_time = time.time()
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        elapsed_time = time.time() - self.start_time
        print(f"time: {elapsed_time:.5f}")
```

```
# Реализация с использованием contextlib
@contextmanager
```

```
def cm_timer_2():
    start_time = time.time()
    try:
        yield
    finally:
        elapsed_time = time.time() - start_time
        print(f"time: {elapsed_time:.5f}")
```

```
if __name__ == "__main__":
    # Тестирование
    print("Тест cm_timer_1:")
    with cm_timer_1():
        time.sleep(1.5)

    print("Тест cm_timer_2:")
    with cm_timer_2():
        time.sleep(0.5)
```

```
`lab_python_fp/field.py`:
```

```

"""
Генератор для последовательного выдачи значений ключей словаря
"""

from typing import Generator

def field(items: list[dict], *args) -> Generator[any, None, None]:
    """
    Генератор для последовательного выдачи значений ключей словаря

    Args:
        items: список словарей
        *args: ключи для извлечения

    Yields:
        значения или словари в зависимости от количества аргументов
    """
    assert len(args) > 0, "Необходимо указать хотя бы один аргумент"

    for dict_item in items:
        if len(args) == 1:
            # Если один аргумент - возвращаем значение
            key = args[0]
            if key in dict_item and dict_item[key] is not None:
                yield dict_item[key]
        else:
            # Если несколько аргументов - возвращаем словарь
            result = {}
            has_values = False
            for key in args:
                if key in dict_item and dict_item[key] is not None:
                    result[key] = dict_item[key]
                    has_values = True

            if has_values:
                yield result

if __name__ == "__main__":
    # Тестирование
    goods = [
        {"title": "Ковер", "price": 2000, "color": "green"},
        {"title": "Диван для отдыха", "color": "black"},
        {"title": None, "price": 5000},
        {"price": 3000, "color": "white"},
    ]

    print("Тест field с одним аргументом:")
    for item in field(goods, "title"):
        print(item)

```

```

print("\nТест field с несколькими аргументами:")
for item in field(goods, "title", "price"):
    print(item)

```

`lab\_python\_fp/gen\_random.py`:

```

"""
Генератор случайных чисел в заданном диапазоне
"""

import random
from typing import Generator

def gen_random(num_count: int, begin: int, end: int) -> Generator[int, None,
None]:
    """
    Генератор случайных чисел в заданном диапазоне

    Args:
        num_count (int): количество чисел
        begin (int): начало диапазона
        end (int): конец диапазона

    Yields:
        случайные числа
    """
    for _ in range(num_count):
        yield random.randint(begin, end)

if __name__ == "__main__":
    # Тестирование
    print("Тест gen_random:")
    for num in gen_random(5, 1, 3):
        print(num, end=" ")
    print()

```

`lab\_python\_fp/print\_result.py`:

```

"""
Декоратор для вывода результата выполнения функции
"""

def print_result(func: callable):
    """

```

Декоратор для вывода результата выполнения функции

```
"""

def wrapper(*args, **kwargs):
    result = func(*args, **kwargs)
    print(func.__name__)

    if isinstance(result, list):
        for item in result:
            print(item)
    elif isinstance(result, dict):
        for key, value in result.items():
            print(f"{key} = {value}")
    else:
        print(result)

    return result

return wrapper


@print_result
def test_1():
    return 1


@print_result
def test_2():
    return "iu5"


@print_result
def test_3():
    return {"a": 1, "b": 2}


@print_result
def test_4():
    return [1, 2]


if __name__ == "__main__":
    print("!!!!!!!")
    test_1()
    test_2()
    test_3()
    test_4()
```

`lab\_python\_fp/process\_data.py`:

```

"""
Обработка данных из JSON файла
"""

import json
import sys

from .gen_random import gen_random
from .unique import Unique
from .print_result import print_result
from .cm_timer import cm_timer_1

# Получаем путь к файлу из аргументов командной строки
path = sys.argv[1] if len(sys.argv) > 1 else "data_light.json"

with open(path, encoding="utf-8") as f:
    data = json.load(f)

@print_result
def f1(arg):
    # Сортированный список профессий без повторений (игнорируя регистр)
    return sorted(
        list(Unique(map(lambda x: x["job-name"], arg), ignore_case=True)),
        key=lambda x: x.lower(),
    )

@print_result
def f2(arg):
    # Фильтрация элементов, начинающихся со слова "программист"
    return list(filter(lambda x: x.lower().startswith("программист"), arg))

@print_result
def f3(arg):
    # Добавление строки "с опытом Python"
    return list(map(lambda x: f"{x} с опытом Python", arg))

@print_result
def f4(arg):
    # Генерация зарплат и объединение с профессиями
    salaries = list(gen_random(len(arg), 100000, 200000))
    return [f"{job}, зарплата {salary} руб." for job, salary in zip(arg, salaries)]

if __name__ == "__main__":
    with cm_timer_1():
        f4(f3(f2(f1(data))))

```

`lab\_python\_fp/sort.py`:

"""

Сортировка списка по абсолютному значению в обратном порядке

"""

```
data = [4, -30, 30, 100, -100, 123, 1, 0, -1, -4]
```

```
if __name__ == "__main__":
```

```
    # Без lambda
```

```
    result = sorted(data, key=abs, reverse=True)
```

```
    print("Без lambda:", result)
```

```
    # С lambda
```

```
    # Использование lambda здесь избыточно, но это требуется по условию задания
```

```
    result_with_lambda = sorted(data, key=lambda x: abs(x), reverse=True) #
```

```
pylint: disable=unnecessary-lambda
```

```
    print("С lambda:", result_with_lambda)
```

`lab\_python\_fp/unique.py`:

"""

Итератор для удаления дубликатов

"""

```
from typing import Generator
```

```
class Unique(object):
```

```
    """
```

```
    Итератор для удаления дубликатов
```

```
    """
```

```
    def __init__(self, items: list | Generator, **kwargs):
```

```
        """
```

```
        Конструктор итератора
```

```
        Args:
```

```
            items (list | Generator): массив или генератор
```

```
            **kwargs: ignore_case - игнорировать регистр для строк
```

```
        """
```

```
        self.items = iter(items)
```

```
        self.ignore_case: bool = kwargs.get("ignore_case", False)
```

```
        self.seen = set()
```

```
    def __next__(self):
```

```
        while True:
```

```

        next_item = next(self.items)

        # Обработка ignore_case для строк
        if self.ignore_case and isinstance(next_item, str):
            key = next_item.lower()
        else:
            key = next_item

        if key not in self.seen:
            self.seen.add(key)
            return next_item

    def __iter__(self):
        return self

if __name__ == "__main__":
    # Тестирование
    from gen_random import gen_random

    print("Тест Unique с числами:")
    data1 = [1, 1, 1, 1, 1, 2, 2, 2, 2, 2]
    for item in Unique(data1):
        print(item, end=" ")
    print()

    print("Тест Unique со строками (ignore_case=False):")
    data2 = ["a", "A", "b", "B", "a", "A", "b", "B"]
    for item in Unique(data2):
        print(item, end=" ")
    print()

    print("Тест Unique со строками (ignore_case=True):")
    for item in Unique(data2, ignore_case=True):
        print(item, end=" ")
    print()

    print("Тест Unique с генератором:")
    data3 = gen_random(10, 1, 3)
    for item in Unique(data3):
        print(item, end=" ")
    print()

```


Скриншот работы приложения

```
(.venv) cunev@ROGStrixG814JIR:~/VSCode/ParadigmsAndConstructsOfProgrammingLanguages/lab3-4$ python ./main.py
=== Тестирование всех модулей ===

1. Тестирование field:
field(goods, 'title'): ['Ковер', 'Диван для отдыха']
field(goods, 'title', 'price'): [{'title': 'Ковер', 'price': 2000}, {'title': 'Диван для отдыха'}]

2. Тестирование gen_random:
gen_random(5, 1, 3): [1, 3, 1, 2, 1]

3. Тестирование unique:
Unique([1, 1, 2, 2, 3, 3]): [1, 2, 3]

4. Тестирование sort:
Исходные данные: [4, -30, 30, 100, -100, 123, 1, 0, -1, -4]
Отсортировано по модулю: [123, 100, -100, -30, 30, 4, -4, 1, -1, 0]

5. Тестирование print_result:
test_1
1
test_2
iu5
test_3
a = 1
b = 2
test_4
1
2

6. Тестирование cm_timer:
cm_timer_1 (1 секунда):
time: 1.00009
cm_timer_2 (0.5 секунды):
time: 0.50018
```

Рисунок 1 Результат работы main.py